



M Ű E G Y E T E M 1 7 8 2



MyHR.

Employee Management System – Developer Guide

Submission by

Muhammad Ibrahim Shoeb (OZLTV3)



Department of Automation and Applied Informatics
VIK, BME
Budapest, Hungary



[MyHR.](#)

Table of Contents

1. Overview	3
2. Tech Stack and Why	3
3. Folder Structure.....	3
4. Database Schema	4
<i>Performance indexes</i>	<i>5</i>
5. Business Logic.....	5
<i>Authentication and access control</i>	<i>5</i>
<i>Internationalization.....</i>	<i>5</i>
<i>Audit immutability</i>	<i>6</i>
<i>User management and soft delete</i>	<i>6</i>
<i>Other/Misc employees.....</i>	<i>6</i>
<i>Hierarchy rules.....</i>	<i>6</i>
<i>Performance-sensitive screens</i>	<i>6</i>
6. Promotion Race Formula.....	6
7. CSV Import Validation Rules.....	7
8. Functional Requirements.....	7
9. Non-Functional Requirements.....	7
10. Resource Requirements.....	8
11. UML Diagrams.....	9
<i>Use Case Diagram</i>	<i>9</i>
<i>Login Sequence.....</i>	<i>10</i>
<i>Employee Creation Sequence</i>	<i>10</i>
<i>Promotion Approval Sequence.....</i>	<i>11</i>
<i>Commendation Issuance Sequence.....</i>	<i>11</i>
<i>Sanction Issuance Sequence.....</i>	<i>12</i>
<i>Salary Increment Approval Sequence.....</i>	<i>12</i>
12. Future Scope for Thesis Semester	12

1. Overview

MyHR is a standalone offline desktop HRMS written in Python. It uses PySide6 for the UI, SQLAlchemy for persistence, SQLite as a local database, qtawesome icons, dictionary-based internationalization, and Qt QSettings for simple company branding.

2. Tech Stack and Why

Layer	Technology	Reason
Language	Python 3	Rapid development and strong desktop/database libraries.
Desktop UI	PySide6 / Qt	Native widgets, tables, dialogs, layout management, and RTL support.
Database	SQLite	Offline single-file storage with simple backup.
ORM	SQLAlchemy	Structured schema, relationships, indexes, eager loading, and safer queries.
Icons	qtawesome / Font Awesome 5	Consistent professional UI icons.
Excel import	openpyxl with fallback XML parser	Supports XLSX onboarding files.
Settings	Qt QSettings	Stores company name and subtitle outside schema changes.
Docs	python-docx	Generates required Word handover documents.
Performance pattern	Pagination and batch calculations	Keeps employee, dashboard, and history screens responsive as data grows.

3. Folder Structure

```
MyHR/
├── main.py                # Entry point
├── requirements.txt       # Python dependencies
├── myhr.db               # SQLite database (auto-created on first run)
├── docs/
│   ├── media/
│   │   ├── demo.gif      # App walkthrough GIF
│   │   └── screenshots/
│   │       ├── login.png
│   │       ├── dashboard.png
│   │       ├── employees.png
│   │       └── ...etc
│   ├── MyHR_User_Guide.docx # End-user manual
│   └── MyHR_Developer_Guide.docx # Technical handover guide
├── scripts/
│   ├── seed_demo_company.py # Generates a 300-person demo dataset
│   └── generate_docs.py      # Rebuilds documentation
├── MockUI/               # React + Tailwind design reference (not the actual app)
├── src/
│   └── core/
│       ├── i18n.py        # Internationalization (EN / HU / AR)
│       └── app_settings.py # Company branding via QSettings
```

database/	
models.py	# 10 SQLAlchemy models + 1 junction table
connection.py	# DB init, business logic, audit helpers
ui/	
styles.py	# Shared style constants
login_window.py	# Login with language selector
main_window.py	# Sidebar navigation shell
assets/	
chevron_down.svg	# Dropdown arrow icon
pages/	
dashboard.py	# Stats, salary increment approval, activity feed
employees.py	# List, add, edit, profile, delete
hierarchy.py	# Org tree (unlimited depth)
promotions.py	# Race tracker, approvals, rules, history
commendations.py	# Single + team awards (3 categories)
sanctions.py	# Disciplinary actions (1–12 month delays)
audit_log.py	# Immutable activity log
import_data.py	# CSV/XLSX bulk import with validation
settings.py	# Salary, increments, users, security, backup

Area	Responsibility
main.py	Application entry point, database initialization, global Qt styling, login startup.
src/core	Internationalization and persisted company settings.
src/database/models.py	SQLAlchemy table definitions and relationships.
src/database/connection.py	Engine/session setup, seed defaults, auth, ID generation, race math, salary increments, audit helper, Other/Misc helpers.
src/ui/main_window.py	Sidebar navigation, role checks, page cache/refresh, logout.
src/ui/pages	Feature page implementations.

4. Database Schema

Table	Columns	Relationships and Notes
system_user	id, username, password_hash, role, full_name, is_active, created_at, last_login	Admin and HR accounts. is_active=False is soft delete.
org_unit	id, name, unit_type, parent_id, head_employee_id, created_at, updated_at	Self-referencing hierarchy. Optional head_employee_id to employee.
title	id, name, label, degree_requirement, base_salary_min, base_salary_max, currency, annual_increment_type, annual_increment_value, promotion_salary_increase_pct, created_at, updated_at	Salary level and increment configuration. Includes L7-L1 and Other.
employee	id, employee_id, first_name, last_name, date_of_birth, phone, personal_email, address, degree, work_email, work_phone, position, join_date, base_salary, status,	Core employee record. Links to org_unit, title, and manager employee.

	annual_increment last_applied, profile_picture, linkedin_url, org_unit_id, title_id, reports_to_id, created_at, updated_at	
promotion_rule	id, from_title_id, to_title_id, base_months, is_active, created_at, updated_at	Configured level transition duration.
promotion_history	id, employee_id, from_title_id, to_title_id, approved_by_id, basis, months_taken, notes, promoted_at	Immutable promotion record.
commendation	id, commendation_ref, title, description, category, months_impact, is_team_award, issued_by_id, issued_at	Award record. Affects standard promotion race.
commendation_employee	commendation_id, employee_id	Join table for single and team commendations.
sanction	id, sanction_ref, employee_id, sanction_type, reason, delay_months, issued_by_id, issued_at, resolved_at, is_resolved	Disciplinary record. Active sanctions delay standard promotion race.
salary_increment_history	id, employee_id, approved_by_id, salary_before, salary_after, increment_type, increment_value, applied_at, notes	Annual increment audit history.
audit_log	id, performed_by_id, performed_by_username, performed_by_name, action, target_table, target_id, description, before_value, after_value, performed_at	Immutable action stream with user identity snapshot.

Performance indexes

- employee is indexed by status, employee_id, work_email, title_id, org_unit_id, and reports_to_id.
- org_unit is indexed by parent_id and name.
- promotion_history and salary_increment_history are indexed by employee/date.
- commendation_employee, commendation, sanction, and audit_log have indexes for the list and race queries used by the UI.

5. Business Logic

Authentication and access control

verify_login hashes the password, checks active system_user rows, updates last_login, and returns a detached UserSession. main_window.py hides and blocks Settings for HR Officers. Employees are data subjects only and never log in.

Internationalization

i18n.py stores English, Hungarian, and Arabic dictionaries. set_language changes the session language; t(key, **kwargs) resolves UI text. Arabic switches the Qt layout direction to right-to-left. Internal enum values, audit action codes, import headers, export headers, and client-entered data are not translated.

Audit immutability

log_action creates audit_log rows with performed_by_id plus performed_by_username and performed_by_name snapshots. The Audit Log page displays username: full name. Renaming a user does not rewrite old audit rows.

User management and soft delete

Admin can edit the admin account and manage HR accounts. HR deletion sets is_active=False instead of deleting the row, preserving historical audit joins and preventing unknown users in the audit UI.

Other/Misc employees

degree_to_title_name maps Other to title Other. ensure_others_org_unit creates the OTHERS branch. valid_other_manager_ids limits managers to the OTHERS head, CEO, or dedicated Other employee handlers. Other employees receive annual increments but have no main promotion track and are excluded from commendation and sanction selection.

seed_demo_company.py creates an OTHERS branch with an Other Employees Head reporting to the CEO, then adds Janitor, Cleaner, Window Cleaner, Security Guard, and Painter examples below that head.

Hierarchy rules

The hierarchy enforces exactly one organization and parent order organization -> division -> department -> unit -> team -> position. Disabled dropdown options and save-time validation prevent invalid structures and parent cycles.

Performance-sensitive screens

Dashboard and Promotion Eligibility use calculate_months_remaining_batch to preserve the same formula as calculate_months_remaining while loading rules, commendations, sanctions, titles, and promotion history once per list refresh. Employee Management uses database-side filtering and 50-row pagination. Promotion History lazy-loads only when its tab is opened and renders 25 rows per page.

6. Promotion Race Formula

race_start = last_promotion_date or employee.join_date

months_elapsed = full_months_between(race_start, today)

commendation_reduction = sum(abs(months_impact)) for commendations since race_start

sanction_addition = sum(delay_months) for unresolved sanctions since race_start

raw_remaining = base_months - months_elapsed - commendation_reduction + sanction_addition

months_remaining = max(0, raw_remaining)

eligible = months_remaining == 0

progress_pct = clamp((months_elapsed + commendation_reduction - sanction_addition) / base_months * 100, 0, 100)

calculate_sub_race builds yearly L7.1/L7.2 style milestones plus the final promotion step. Other employees get ongoing Other.1, Other.2, and so on for annual increments only. Promotion approval applies the next title's promotion_salary_increase_pct to current salary and creates PromotionHistory. Annual increments are separate and create SalaryIncrementHistory.

The batch race calculator is verified against the single-employee calculator and must remain mathematically identical. It exists only to avoid repeated queries on list screens.

7. CSV Import Validation Rules

- Required: first_name, last_name, department, degree, position, join_date, base_salary.
- Header aliases normalize common names such as email to work_email and salary to base_salary.
- Degree must be BSc, MSc, PhD, or Other.
- Join date must be YYYY-MM-DD and not in the future.
- Base salary must be numeric and greater than zero, then must pass configured salary range validation during import.
- Status defaults to active and must be active, inactive, or on_leave.
- Work email is format-checked, unique against the database, and unique inside the file.
- manager_work_email is format-checked and wired after rows are created.
- commendation_months must be 0, 1, 3, or 6.
- active_sanction_months must be 0 to 12.
- sanction_type accepts verbal warning, written warning, suspension, or final warning when sanction months exist.

8. Functional Requirements

- Authenticate Admin and HR Officer users.
- Support English, Hungarian, and Arabic UI with RTL for Arabic.
- Manage employees, hierarchy, promotions, commendations, sanctions, audit logs, and imports.
- Configure salary ranges, promotion rules, and annual increment rules.
- Support Other/Misc employees with OTHERS hierarchy and annual increments only.
- Calculate promotion eligibility live and show main/sub-race status.
- Approve promotions and annual increments with history records.
- Create, edit, deactivate, and reactivate HR accounts.
- Export employees and back up the SQLite database.
- Log important actions with immutable audit identity snapshots.
- Paginate employee and promotion history tables so the UI remains usable with thousands of records.

9. Non-Functional Requirements

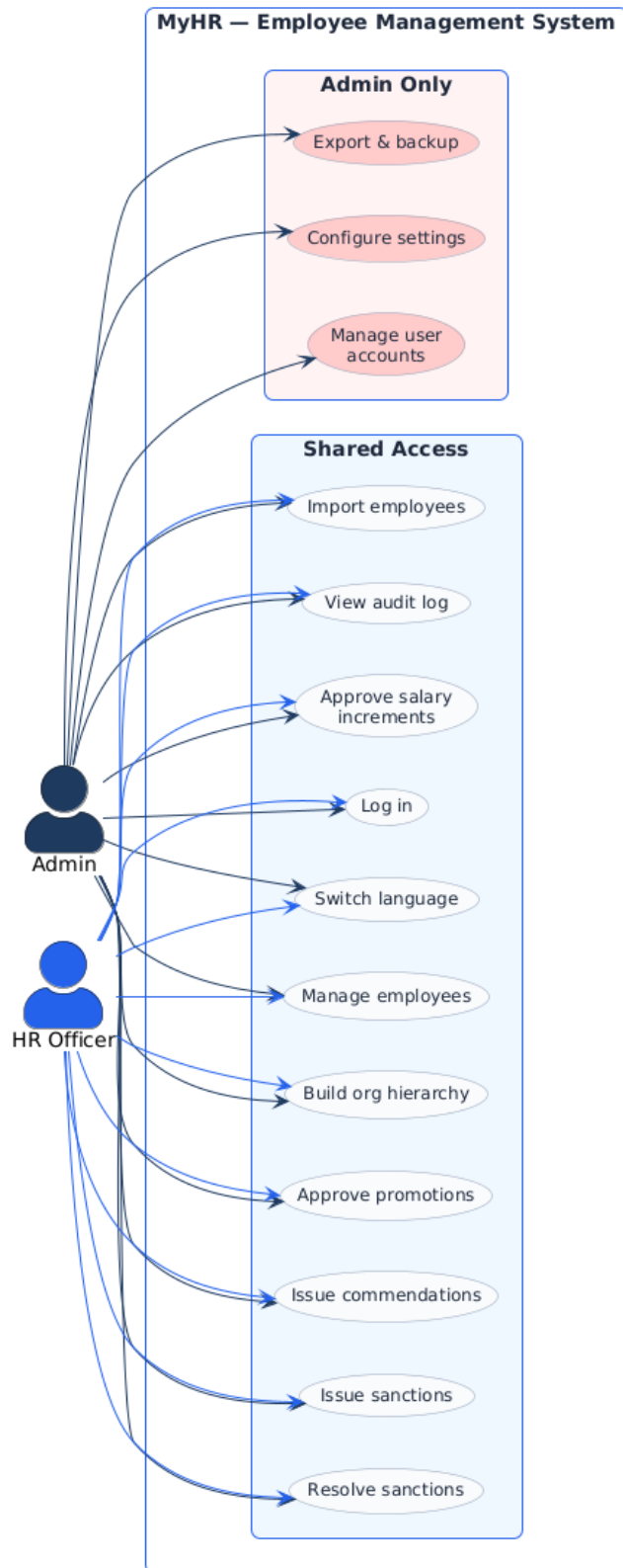
- Offline-first operation without a runtime server.
- Professional and consistent PySide6 UI.
- Role-based access control.
- Data integrity through ORM relationships and validation before writes.
- Auditability with user snapshots.
- Localization without changing client data or internal contracts.
- Simple backup and export for operational safety.
- Bounded rendering and indexed queries for larger employee datasets.

10. Resource Requirements

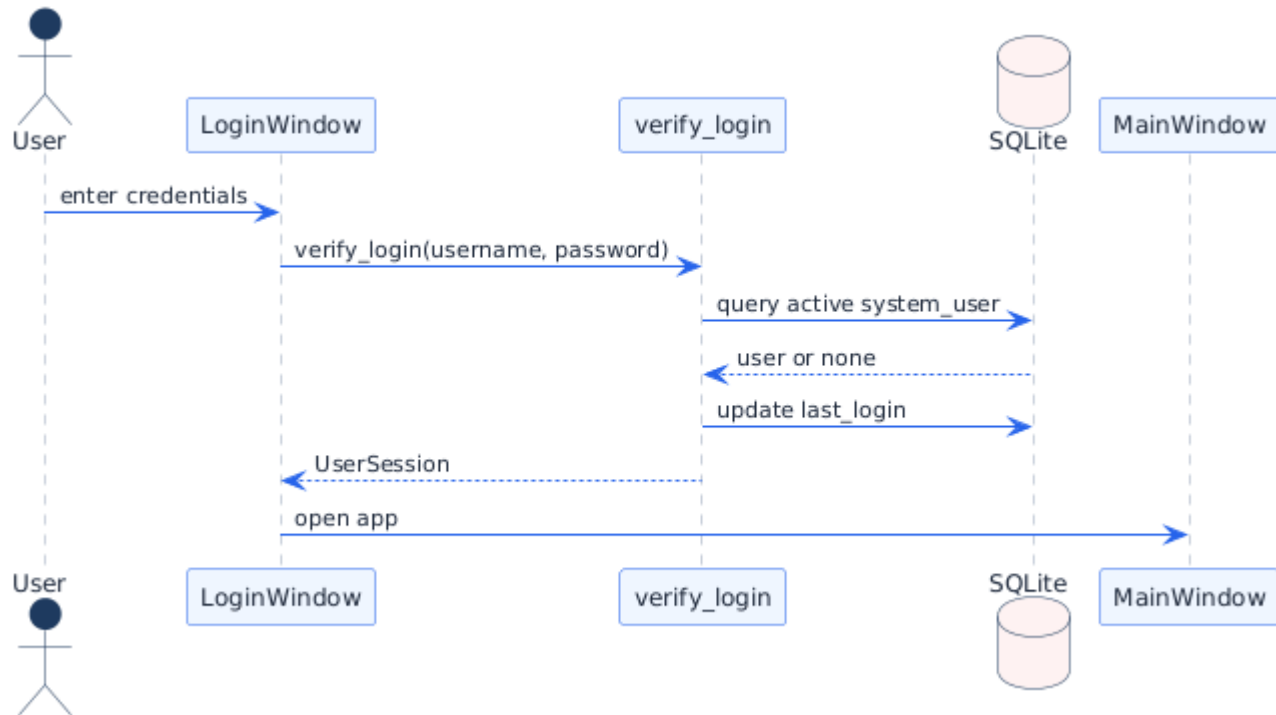
Resource	Requirement
OS	Windows development environment; PySide6 can be tested on other desktop OSes.
Python	Python 3.12+ recommended.
RAM	4 GB minimum recommended.
Storage	Project files, SQLite database, exports, and backups.
Dependencies	PySide6, SQLAlchemy, qtawesome, openpyxl, python-docx.
Network	Only needed to install dependencies. Runtime is offline.

11. UML Diagrams

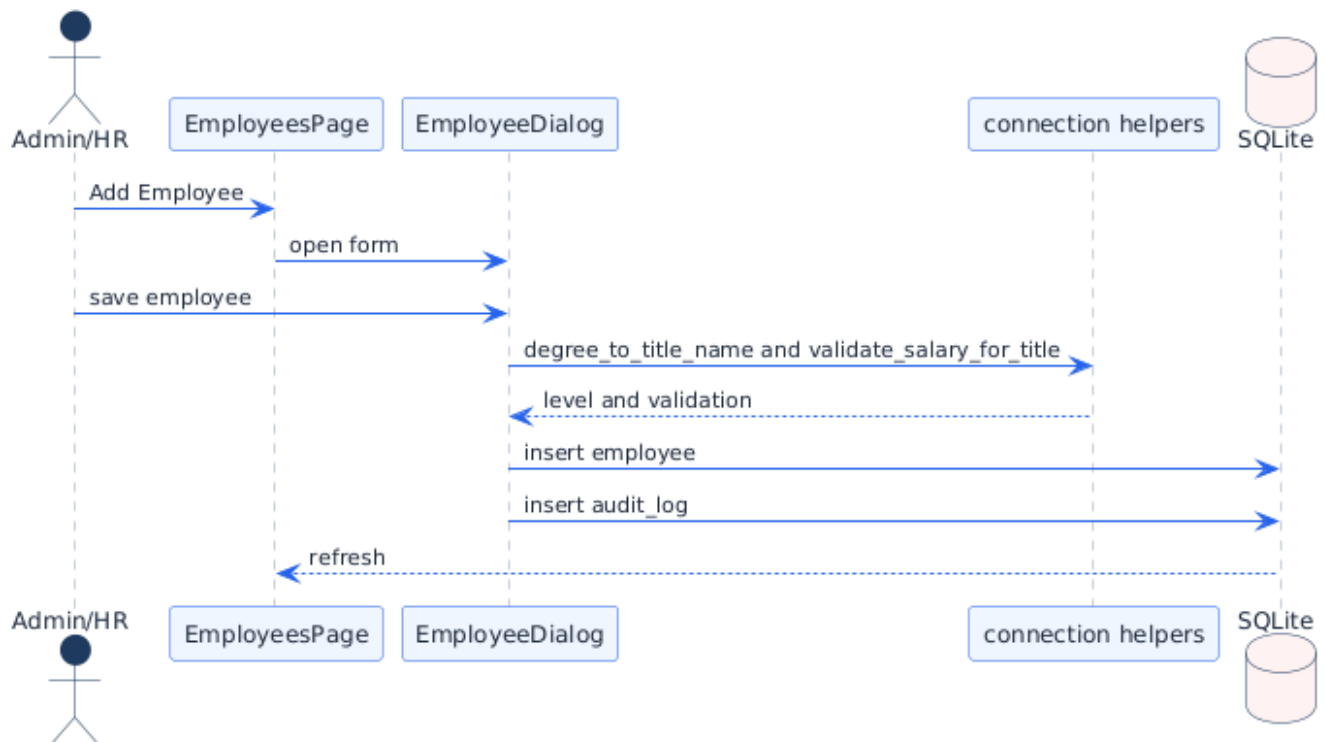
Use Case Diagram



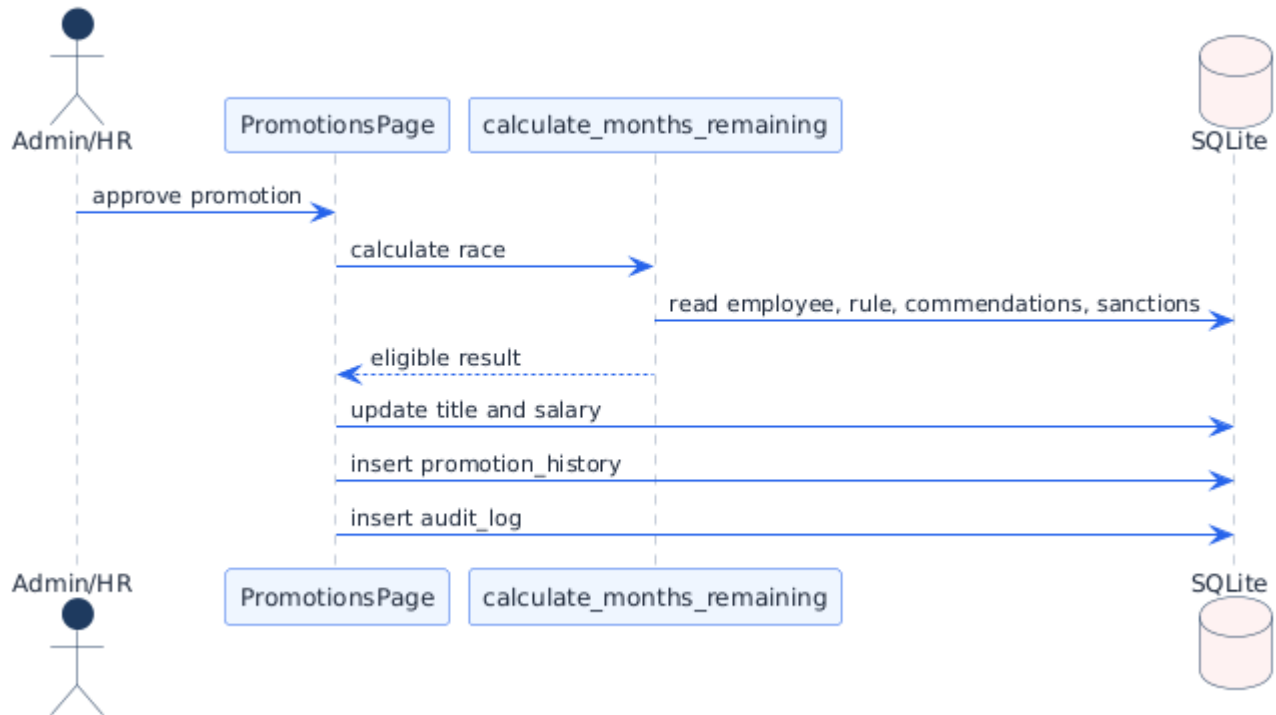
Login Sequence



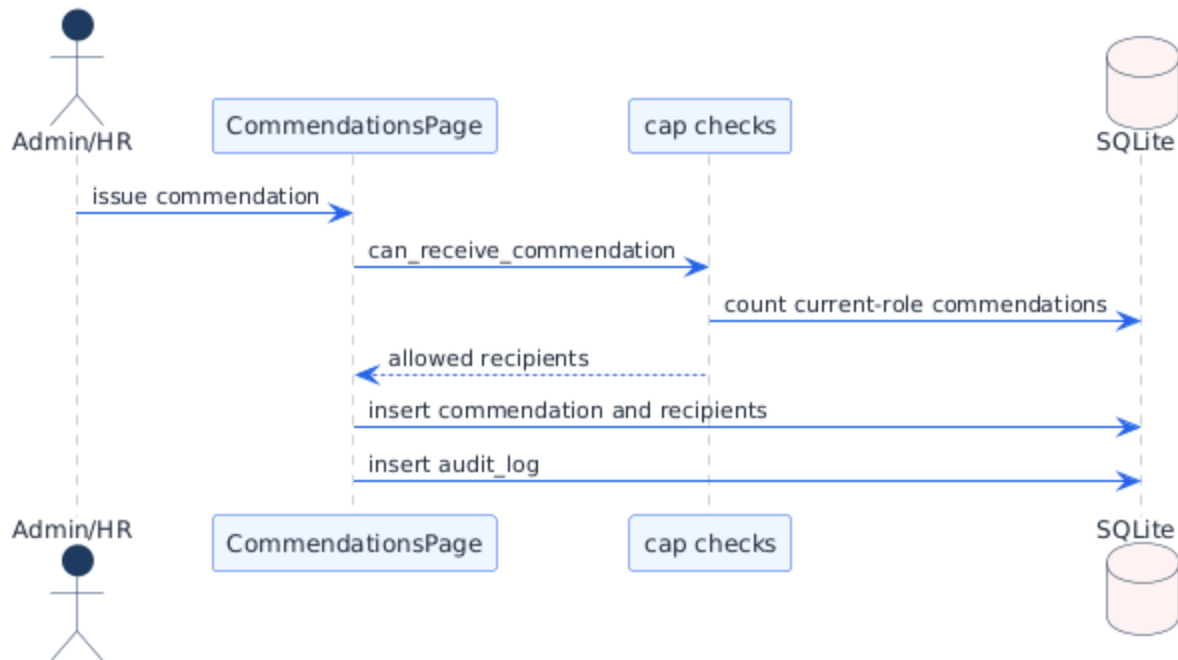
Employee Creation Sequence



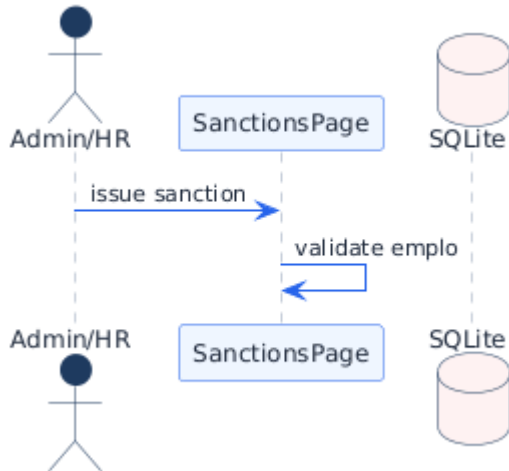
Promotion Approval Sequence



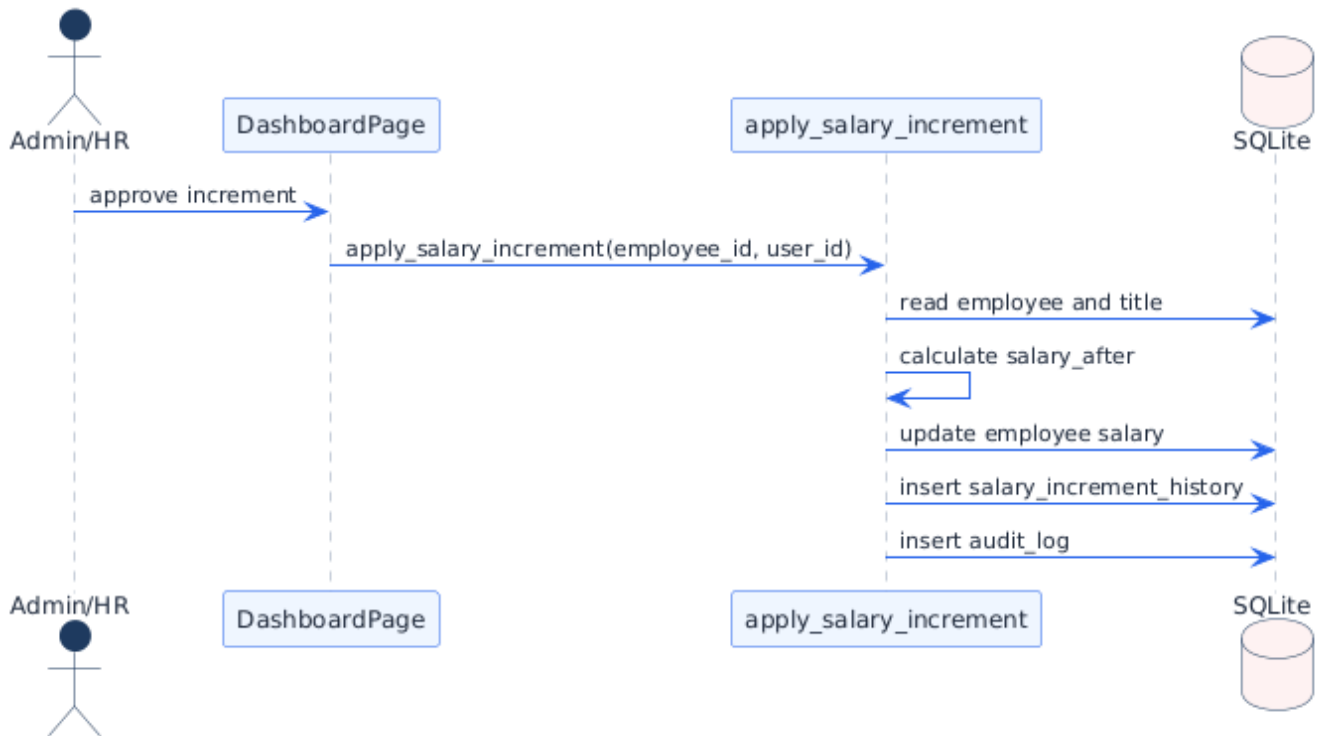
Commendation Issuance Sequence



Sanction Issuance Sequence



Salary Increment Approval Sequence



12. Future Scope for Thesis Semester

- Package the desktop app into a user-friendly installer.
- Add automated tests for race math, import validation, access control, user management, audit snapshots, and performance-sensitive list pages.
- Add richer reports for yearly promotions, salary budget impact, sanctions, commendations, and departments.
- Add scheduled backup reminders and backup health checks.
- Introduce Alembic or another migration tool if schema changes become frequent.
- Consider encrypted backups or protected local storage for sensitive HR data.

- Extend roles only if the organization needs more than Admin and HR Officer.